

Agent Harnessing — Concepts and Python Implementation

Table of Contents

1. [What Is an Agent Harness?](#)
 2. [The Core Loop: ReAct](#)
 3. [Harness vs. Framework vs. Runtime](#)
 4. [Anatomy of a Harness](#)
 5. [Building a Minimal Harness in Python](#)
 6. [Adding Guardrails and Budget Enforcement](#)
 7. [Memory and State Persistence](#)
 8. [Failure Handling](#)
 9. [Production Harnesses: LangGraph and Beyond](#)
 10. [Practical Notes](#)
 11. [Summary](#)
 12. [Sources](#)
-

1. What Is an Agent Harness?

An agent harness is the software infrastructure that wraps around a language model and turns it from a stateless text-in/text-out function into something that can actually complete multi-step, real-world tasks. It is the software scaffolding — tools, memory, sandboxes, and feedback loops — that turns a model into an agent. ¹

The distinction is worth internalizing precisely: **the harness is everything except the model's actual reasoning**. It manages tool execution, memory storage, state persistence, and error recovery, acting as the intermediary between the LLM and the outside world. ² A widely repeated framing from LangChain captures it in one line: *"If you're not the model, you're the harness."* ³

A concrete example makes this tangible: asked to fix a failing test in a Python repo, a bare model can only write text describing a plausible fix — it cannot read the test file, run `pytest`, observe the real error output, edit the failing function, or confirm the fix actually passed. With a harness wired around it, that same model can execute the full read → run → observe → edit → verify loop and produce a real, verified result. ⁴

2. The Core Loop: ReAct

Most harnesses implement some version of the same cycle, commonly called **ReAct** (Reason + Act):

- 1. **Reason** — the model reads everything in its context (task, memory, previous results) and decides what action to take next.
- 2. **Act** — the harness carries out that action: running a tool, executing code in a sandbox, calling an API, or writing to storage.
- 3. **Observe** — the harness captures the result and feeds it back to the model as new context.
- 4. **Repeat** — the model uses that result to decide what to do next, until the task is complete.¹

The model only ever does step 1. Steps 2-4 — execution, observation, and control flow — are entirely the harness's responsibility. Harnesses can only execute the tools they have logic for, which is why many production harnesses give the agent one general-purpose tool (such as `bash`) rather than trying to hand-build a bespoke tool for every possible action.⁵

3. Harness vs. Framework vs. Runtime

These three terms get used loosely in practice, but the useful distinction is:

Term	What it actually is
Harness	The runtime configuration of supports actually exposed to the agent while it runs — tools, memory, guardrails, observability, wired together for a specific agent ⁶
Framework	Infrastructure for <i>composing</i> agents and tools (e.g. LangGraph, CrewAI) — a framework is what you use to <i>build</i> a harness, not the harness itself ⁶
Agent-Computer Interface (ACI)	Specifies <i>how</i> an agent acts through tools — one component that lives inside a harness, not the whole thing ⁶

Frameworks like LangGraph, or products like LangChain's Deep Agents, are essentially pre-built harnesses so you don't have to hand-roll every piece yourself. LangChain Deep Agents, for instance, is built on LangGraph as its runtime and ships with a planning tool, a virtual filesystem, subagent spawning, automatic context compaction, and human-in-the-loop middleware out of the box.³

4. Anatomy of a Harness

A harness that actually holds up in production is generally built from five components, each with a single responsibility:⁷

Component	Responsibility
Tool registry	Maintains a catalog of callable functions, validates arguments against a schema, executes them safely, returns structured results
Memory management	Stores conversation history, injects relevant context, compresses old messages to stay within context limits
Guardrail enforcement	Limits token budgets, caps tool-call counts, prevents infinite loops, blocks dangerous actions
Failure handling	Retries on transient errors, degrades gracefully when a tool is unavailable, escalates to human review when confidence is low
State persistence	Logs each turn and tool invocation so the session survives restarts and is auditable afterward

Without a harness, you have a stateless API call. With one, you have a system.⁷

5. Building a Minimal Harness in Python

A minimal harness needs three things: a model interface, a tool registry, and a loop.⁷ This uses the Anthropic Messages API directly — no orchestration framework, no vector database, no external state store required for the minimal version.⁸

5.1 Setup

```
bash
pip install anthropic
```

```
bash
export ANTHROPIC_API_KEY="your-api-key"
```

5.2 Tool Registry

```
python
```

```

from dataclasses import dataclass
from typing import Callable, Any

@dataclass
class Tool:
    name: str
    description: str
    input_schema: dict
    func: Callable[..., Any]

class ToolRegistry:
    def __init__(self):
        self._tools: dict[str, Tool] = {}

    def register(self, tool: Tool) -> None:
        self._tools[tool.name] = tool

    def get_schemas(self) -> list[dict]:
        """Return tool definitions in the shape the model API expects."""
        return [
            {
                "name": t.name,
                "description": t.description,
                "input_schema": t.input_schema,
            }
            for t in self._tools.values()
        ]

    def execute(self, name: str, tool_input: dict) -> dict:
        if name not in self._tools:
            return {"error": f"unknown tool: {name}"}
        try:
            result = self._tools[name].func(**tool_input)
            return {"result": result}
        except Exception as e:
            # Structured errors, not raw stack traces, so the model can react to them
            return {"error": str(e), "tool": name}

```

5.3 Registering Tools

python

```
def read_file(path: str) -> str:
    with open(path) as f:
        return f.read()

def run_tests(path: str) -> str:
    import subprocess
    result = subprocess.run(["pytest", path, "-v"], capture_output=True, text=True)
    return result.stdout + result.stderr

registry = ToolRegistry()
registry.register(Tool(
    name="read_file",
    description="Read the contents of a file at the given path.",
    input_schema={"type": "object", "properties": {"path": {"type": "string"}}},
    func=read_file,
))
registry.register(Tool(
    name="run_tests",
    description="Run pytest against the given path and return the output.",
    input_schema={"type": "object", "properties": {"path": {"type": "string"}}},
    func=run_tests,
))
```

5.4 The Agent Loop

python

```
import anthropic

client = anthropic.Anthropic()

class AgentHarness:
    def __init__(self, registry: ToolRegistry, model: str = "claude-sonnet-4-5"
        self.registry = registry
        self.model = model
        self.max_turns = max_turns
        self.history: list[dict] = []

    def run(self, task: str) -> str:
        self.history.append({"role": "user", "content": task})

        for turn in range(self.max_turns):
            response = client.messages.create(
                model=self.model,
                max_tokens=2048,
                tools=self.registry.get_schemas(),
                messages=self.history,
            )
            self.history.append({"role": "assistant", "content": response.content[0].text})

            if response.stop_reason != "tool_use":
                # Model produced a final answer – extract and return it
                return "".join(b.text for b in response.content if b.type == "text")

            # Execute every tool call the model requested this turn
            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    outcome = self.registry.execute(block.name, block.input)
                    tool_results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": str(outcome.get("result", outcome.get("error", ""))),
                        "is_error": "error" in outcome,
                    })
            self.history.append({"role": "user", "content": tool_results})

        return "Max turns reached without a final answer."
```

5.5 Running It

```
python

harness = AgentHarness(registry)
result = harness.run("Read test_math.py, run its tests, and summarize any failure")
print(result)
```

This is the whole loop: reason (model call) → act (tool execution) → observe (result appended to history) → repeat, bounded by `(max_turns)` as the first, crudest guardrail against an infinite loop.

6. Adding Guardrails and Budget Enforcement

The minimal loop above has exactly one safety mechanism (`(max_turns)`). Production harnesses limit token budgets, cap tool-call counts, prevent infinite loops, and block dangerous actions outright.⁷

```
python

class BudgetExceeded(Exception):
    pass

class GuardedHarness(AgentHarness):
    def __init__(self, *args, max_tool_calls: int = 30, blocked_tools: set[str]):
        super().__init__(*args, **kwargs)
        self.max_tool_calls = max_tool_calls
        self.tool_call_count = 0
        self.blocked_tools = blocked_tools or set()

    def _execute_guarded(self, name: str, tool_input: dict) -> dict:
        if name in self.blocked_tools:
            return {"error": f"tool '{name}' is not permitted in this session"}
        self.tool_call_count += 1
        if self.tool_call_count > self.max_tool_calls:
            raise BudgetExceeded(f"exceeded {self.max_tool_calls} tool calls")
        return self.registry.execute(name, tool_input)
```

For irreversible or high-risk actions (deleting data, sending external messages, executing arbitrary shell commands), the standard pattern is a **human-in-the-loop pause**: the harness detects the action, halts the loop, and waits for explicit approval before executing — rather than letting the agent take the action autonomously.⁹

python

```
IRREVERSIBLE_TOOLS = {"delete_record", "send_email", "execute_shell"}

def requires_approval(tool_name: str) -> bool:
    return tool_name in IRREVERSIBLE_TOOLS
```

7. Memory and State Persistence

Models have no additional knowledge beyond their weights and what's in their current context — without access to edit model weights, the only way to add knowledge mid-task is context injection, and the filesystem (or an equivalent durable store) is the core primitive for that.⁵

A minimal file-backed session store:

python

```
import json
from pathlib import Path

class SessionStore:
    def __init__(self, session_dir: str = ".harness_sessions"):
        self.dir = Path(session_dir)
        self.dir.mkdir(exist_ok=True)

    def save(self, session_id: str, history: list[dict]) -> None:
        (self.dir / f"{session_id}.json").write_text(json.dumps(history))

    def load(self, session_id: str) -> list[dict]:
        path = self.dir / f"{session_id}.json"
        return json.loads(path.read_text()) if path.exists() else []
```

Wire it into the harness so a session survives a process restart:

python

```

class PersistentHarness(GuardedHarness):
    def __init__(self, *args, store: SessionStore, session_id: str, **kwargs):
        super().__init__(*args, **kwargs)
        self.store = store
        self.session_id = session_id
        self.history = self.store.load(session_id)

    def run(self, task: str) -> str:
        result = super().run(task)
        self.store.save(self.session_id, self.history)
        return result

```

As history grows, most production harnesses also apply **context compaction** — summarizing or dropping older turns once the conversation approaches the model's context window limit — so long-running sessions don't silently degrade or fail once history exceeds what fits in a single call.³

8. Failure Handling

Structured, harness-level failure handling — not per-agent, ad hoc `try/except` blocks — is what separates a demo from something production-worthy:

```

python

import time

def execute_with_retry(registry: ToolRegistry, name: str, tool_input: dict, max_retries: int):
    for attempt in range(max_retries + 1):
        outcome = registry.execute(name, tool_input)
        if "error" not in outcome:
            return outcome
        if attempt < max_retries:
            time.sleep(2 ** attempt) # exponential backoff on transient failure
    return outcome # exhausted retries - surface the structured error to the m

```

The philosophy worth adopting: harness engineering treats every agent failure as a system problem to permanently fix, not a prompt to simply retry² — a tool that fails the same way repeatedly is a signal to fix the tool or its schema, not to keep re-asking the model to try harder.

9. Production Harnesses: LangGraph and Beyond

Hand-rolling a harness (as above) is valuable for understanding the mechanics, but production systems generally build on an existing framework rather than maintaining the loop, retries, and state persistence themselves. A LangGraph-based harness typically layers:

- **Nodes** as execution units (an LLM call, a tool call, a reflection step)
- **Conditional edges** as the control flow that decides what runs next — including routing to a human-approval node for irreversible actions
- **A shared `State` object** that carries history, budgets, and intermediate results between nodes⁹

```
python

from langgraph.graph import StateGraph, END
from typing import TypedDict

class HarnessState(TypedDict):
    task: str
    history: list[dict]
    tool_call_count: int
    max_tool_calls: int

def agent_node(state: HarnessState) -> HarnessState:
    # model call + tool dispatch, updating state["history"] and state["tool_cal
    ...
    return state

def should_continue(state: HarnessState) -> str:
    if state["tool_call_count"] >= state["max_tool_calls"]:
        return END
    return "agent" if not_done(state) else END

workflow = StateGraph(HarnessState)
workflow.add_node("agent", agent_node)
workflow.set_entry_point("agent")
workflow.add_conditional_edges("agent", should_continue)
app = workflow.compile()
```

This is functionally the same five-component anatomy from Section 4 — tool registry, memory, guardrails, failure handling, state persistence — just expressed as a graph instead of a hand-written `while` loop. It's also directly applicable to a multi-agent pipeline shaped

like `enrichment_worker → reflection_agent → synthesizer`; each agent is a node, the reflection agent's reroute decision is a conditional edge, and the shared `State` object is exactly what carries evidence, confidence scores, and retry counts between them.

For a fuller reference architecture, production harness write-ups commonly combine LangGraph for orchestration, FastAPI for serving, a tracing/observability layer (e.g. LangSmith or Langfuse) for auditability, and a vector-capable store (e.g. PostgreSQL with pgvector) for long-term memory.¹⁰

10. Practical Notes

1. **Start minimal.** The guidance worth taking seriously: start with the simplest approach possible and only add moving parts (memory stores, retry policies, subagents) when the task genuinely needs them — the harness doesn't make you smarter about what to build.⁴
 2. **You don't always need one.** Simple, single-turn tasks don't require a harness at all; any multi-step or long-running work does.²
 3. **The model is pluggable.** A well-built harness treats the model as a swappable component — multiple models can share the same harness, which matters if you ever want to A/B test model versions without rewriting orchestration logic.²
 4. **Guardrails belong in the harness, not the prompt.** Token budgets, tool-call caps, and blocked actions should be enforced in code, not requested politely in a system prompt — prompts are not a reliable enforcement mechanism.
 5. **Production harnesses rarely allow autonomous destructive actions.** Deleting data, sending irreversible messages, or executing arbitrary shell commands should route through a human-in-the-loop pause rather than executing automatically.⁹
 6. **State persistence enables auditability, not just resumability.** A logged history of every turn and tool call is what lets you reconstruct *why* an agent did something after the fact — treat it as a debugging and compliance asset, not just a resume-session convenience.
-

11. Summary

Need	Harness Component
Let the model actually act on the world	Tool registry + safe execution
Keep context coherent across a long task	Memory management + compaction
Prevent runaway cost or infinite loops	Guardrails: token/tool-call budgets
Survive transient errors gracefully	Structured retry + failure handling
Resume sessions, audit what happened	State persistence
Gate irreversible actions	Human-in-the-loop approval nodes

A harness is what separates "a model that can generate a plausible-looking fix" from "a system that reads the failing test, runs it, edits the code, and verifies the fix actually passes." The model supplies reasoning; everything else — execution, memory, safety, and auditability — is harness engineering, whether you hand-roll it as a Python loop or express it as a LangGraph `StateGraph`.

12. Sources

Note: "agent harness" is an emerging, still-settling term (it gained wide use starting around early 2026) — several sources above are practitioner blog posts rather than formal specifications, so terminology and exact component boundaries vary somewhat by author. Treat the code in this document as an illustrative reference implementation, not a production-hardened library.

1. [What is an AI Agent Harness? — Databricks Blog](#) ↩ ↩²
2. [What Is an Agent Harness? The Infrastructure That Makes AI Agents Actually Work — Firecrawl](#) ↩ ↩² ↩³ ↩⁴
3. [What Is an Agent Harness? A Beginner's Guide — DataCamp](#) ↩ ↩² ↩³
4. [What Is an Agent Harness? How AI Agents Get Tools, Memory, and Control — Medium](#) ↩ ↩²
5. [The Anatomy of an Agent Harness — LangChain Blog](#) ↩ ↩²
6. [AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents — arXiv](#) ↩ ↩² ↩³
7. [Building an AI Agent Harness from Scratch: The Architecture Between LLM and Agent — DEV Community](#) ↩ ↩² ↩³ ↩⁴
8. [How to Build a Minimal Agent Harness in Python: Step-by-Step with Session Persistence — MindStudio](#) ↩

9. The Anatomy of an Agent Harness: A Complete Guide — QubitTool ↩ ↩² ↩³
10. Building a Production-Ready AI Agent Harness — DEV Community ↩